A Catalog of Lattice Structures for Propagator Networks

Prologos Research

2026-03-14

Contents

1 Introduction **3**

2 Lattice Taxonomy **4**
2.1 Partial Orders (Posets) 4
2.2 Semilattices 4
2.2.1 Join-Semilattice 4
2.2.2 Meet-Semilattice 5
2.3 Lattices 5
2.3.1 Bounded Lattice 5
2.3.2 Distributive Lattice 5
2.3.3 Modular Lattice 5
2.3.4 Complemented Lattice 5
2.3.5 Boolean Algebra (Boolean Lattice) 6
2.4 Complete Lattices 6
2.4.1 Algebraic Lattice 6
2.4.2 Continuous Lattice 6
2.5 Heyting Algebras 6
2.6 Bilattices 7
2.7 Quantales 7

3 Concrete Lattice Instances **7**
3.1 Flat Lattices 7
3.2 Two-Point Lattice (Bool) 8
3.3 Three-Point Lattices 8
3.3.1 Definedness Lattice 8
3.3.2 Belnap's FOUR (Three-Point on Each Axis) 8

3.4 Powerset Lattices . . . 9
3.5 Interval Lattices . . . 9
3.5.1 Numeric Intervals . . . 9
3.5.2 Universe Level Intervals . . . 9
3.6 Map/Dictionary Lattices . . . 9
3.7 Sign Lattice . . . 10
3.8 Parity Lattice . . . 10
3.9 Type Lattice . . . 10
3.10 Session Type Lattice . . . 11
3.11 QTT Quantity Lattice . . . 11
3.12 Substitution Lattice . . . 12
3.13 Herbrand Interpretation Lattice . . . 12
3.14 Constraint Store Lattice . . . 12
3.15 ATMS Environment Lattice . . . 12

4 Lattice Constructions and Compositions 13
4.1 Product Lattice . . . 13
4.2 Reduced Product . . . 13
4.3 Sum (Coproduct) Lattices . . . 14
4.3.1 Disjoint Sum . . . 14
4.3.2 Coalesced Sum . . . 14
4.4 Lifting . . . 14
4.5 Smash Product . . . 14
4.6 Function Space Lattice . . . 15
4.7 Powerdomain Constructions . . . 15
4.8 Galois Connections . . . 15
4.8.1 Properties . . . 16
4.8.2 Galois Connection Catalog for Prologos . . . 16
4.8.3 Composition of Galois Connections . . . 16
4.9 Lattice Embeddings . . . 16
4.10 Lattice Quotients . . . 17
4.11 Lattice Fixed-Point Operators . . . 17
4.11.1 Knaster-Tarski Theorem . . . 17
4.11.2 Kleene Iteration . . . 17
4.11.3 Widening and Narrowing . . . 17
4.11.4 Stratified Fixed Points . . . 18
4.12 Lattice-Indexed Families . . . 18

5 Combinatorial Lattice Application Map **18**
5.1 Type Inference Engine 18
5.2 QTT Resource Tracking 19
5.3 Session Types 19
5.4 Well-Founded Logic Engine (WFLE) 19
5.5 Abstract Interpretation Framework 19
5.6 ATMS and Search 20
5.7 Propagator Network Infrastructure 20

6 Advanced Composition Patterns **20**
6.1 Lattice Transformers 20
6.2 Fibered Lattices 21
6.3 Lattice Comonads and Demand 21
6.4 Lattice Adjunctions Beyond Galois Connections 22

7 Speculative and Creative Applications **22**
7.1 Lattice of Lattices 22
7.2 Tropical Semiring Lattice 22
7.3 Partition Lattice 23
7.4 Concept Lattice (Formal Concept Analysis) 23
7.5 Information Flow Lattice 23
7.6 Lattice of Regular Languages 23
7.7 Scott Information Systems 23
7.8 Residuated Lattices 24

8 Summary Table: Lattice Applicability to Prologos Subsystems **24**

9 References **25**

1 Introduction

This document catalogs lattice structures relevant to propagator networks, with particular attention to their application in Prologos—a language unifying dependent types, session types, linear types (QTT), logic programming, and propagators. We organize the material into four layers:

1. **Fundamental lattice taxonomy**: the zoo of lattice kinds, from partial orders through complete lattices and beyond.

2. **Concrete lattice instances**: specific lattices that arise in programming language implementation, static analysis, and constraint solving.

3. **Composition and construction**: how lattices combine via products, sums, lifting, Galois connections, embeddings, and functorial mappings.

4. **Applications to Prologos**: how each lattice or composition maps onto a specific subsystem (type inference, QTT resource tracking, session types, well-founded logic, abstract interpretation, ATMS).

Throughout, we use standard notation: $\sqsubseteq$ for the partial order, $\sqcup$ for join (least upper bound), $\sqcap$ for meet (greatest lower bound), $\bot$ for bottom, $\top$ for top.

2 Lattice Taxonomy

2.1 Partial Orders (Posets)

The weakest structure. A set P with a binary relation $\sqsubseteq$ that is reflexive, antisymmetric, and transitive. Not every pair of elements need be comparable (i.e., the order may be partial, not total).

- **Propagator relevance**: Every cell's value domain must be at least a poset. Monotonicity of propagators is defined with respect to this order.

2.2 Semilattices

2.2.1 Join-Semilattice

A poset where every finite non-empty subset has a least upper bound (join). This is the **minimal requirement** for a propagator cell's merge operation. The merge laws—commutativity, associativity, idempotency—are precisely the axioms of a join-semilattice.

- **Propagator relevance**: The foundational algebraic structure of the Radul-Sussman propagator model. Every cell's merge function defines a join-semilattice.

2.2.2 Meet-Semilattice

Dual: every finite non-empty subset has a greatest lower bound (meet). Used for descending/narrowing computations.

- **Propagator relevance**: Descending cells in the bilattice infrastructure (`net-new-cell-desc`) use meet to refine from $\top$ downward, eliminating possibilities.

2.3 Lattices

A poset that is simultaneously a join-semilattice and a meet-semilattice: every pair of elements has both a join and a meet.

2.3.1 Bounded Lattice

A lattice with distinguished $\bot$ (bottom/least) and $\top$ (top/greatest) elements. Every finite lattice is bounded. The propagator convention: $\bot$ = no information, $\top$ = contradiction.

2.3.2 Distributive Lattice

A lattice satisfying: $a \sqcup (b \sqcap c) = (a \sqcup b) \sqcap (a \sqcup c)$ and its dual. Distributivity is **not** automatic—the diamond lattice M_3 and the pentagon lattice N_5 are the two minimal non-distributive lattices. Many practical lattices (powersets, flat lattices, intervals over total orders) are distributive.

- **Propagator relevance**: Distributivity permits certain optimizations in constraint propagation; it guarantees that join and meet interact predictably.

2.3.3 Modular Lattice

Weaker than distributive: $a \sqcup (b \sqcap c) = (a \sqcup b) \sqcap c$ whenever $a \sqsubseteq c$. Every distributive lattice is modular. The lattice of subgroups of an abelian group is modular but not necessarily distributive.

2.3.4 Complemented Lattice

A bounded lattice where every element a has a complement $\bar{a}$ such that $a \sqcup \bar{a} = \top$ and $a \sqcap \bar{a} = \bot$. The complement need not be unique unless the lattice is also distributive.

2.3.5 Boolean Algebra (Boolean Lattice)

A complemented distributive lattice. Equivalently, a bounded distributive lattice with a complement operation satisfying De Morgan's laws. The canonical example is the powerset lattice $(\mathcal{P}(S), \subseteq)$.

- **Propagator relevance**: Boolean algebras model classical propositional reasoning, ATMS worldview management, and set-based constraint domains.

2.4 Complete Lattices

Every subset (not just finite ones) has both a join and a meet. This is the setting for the Knaster-Tarski fixed-point theorem: every monotone function on a complete lattice has a least and greatest fixed point.

- **Propagator relevance**: The Knaster-Tarski theorem is the theoretical backbone guaranteeing that `run-to-quiescence` terminates at the least fixed point. Completeness is needed for infinite-domain propagation (e.g., type universes).

2.4.1 Algebraic Lattice

A complete lattice where every element is the join of compact elements below it. The lattice of ideals of a ring is algebraic. Scott domains (from domain theory) are typically algebraic.

2.4.2 Continuous Lattice

A complete lattice where every element is the directed join of elements "way-below" it ($x \ll y$ iff for every directed set D with $y \sqsubseteq \bigsqcup D$, there exists $d \in D$ with $x \sqsubseteq d$). Continuous lattices are the denotational-semantics workhorses.

2.5 Heyting Algebras

A bounded lattice with a relative pseudo-complement operation $a \Rightarrow b$ (the largest c such that $a \sqcap c \sqsubseteq b$). Every Boolean algebra is a Heyting algebra, but not conversely. The open sets of a topological space form a Heyting algebra under inclusion.

- **Propagator relevance**: Heyting algebras model intuitionistic logic, which is the natural logic of type theory. Implications in dependent type checking correspond to Heyting implication.

2.6 Bilattices

An algebraic structure $(B, \leq_t, \leq_k)$ with two lattice orderings:

- $\leq_t$: the **truth ordering** (false $\leq_t$ true)
- $\leq_k$: the **knowledge ordering** (neither $\leq_k$ true, neither $\leq_k$ false, neither $\leq_k$ both)

The four-valued Belnap bilattice **FOUR** has elements: $\bot_k$ (neither true nor false / unknown), **t** (true), **f** (false), $\top_k$ (both true and false / inconsistent).

- **Propagator relevance**: Prologos's `bilattice.rkt` implements exactly this structure. Bilattice variables pair an ascending cell (lower bound, knowledge accumulates) with a descending cell (upper bound, possibilities narrow). The gap between lower and upper yields three-valued readings: true, false, or unknown. This is the foundation of the well-founded logic engine (WFLE).

2.7 Quantales

A complete lattice equipped with an associative binary operation $\otimes$ that distributes over arbitrary joins: $a \otimes (\bigsqcup S) = \bigsqcup\{a \otimes s \mid s \in S\}$. Quantales unify lattice theory with monoid structure.

- **Propagator relevance**: The QTT resource semiring can be viewed through the lens of quantales when it carries a compatible lattice ordering. Resource usage tracking $(0, 1, \omega)$ under the QTT semiring operations (addition distributing over the quantity ordering) has quantale-like structure.

3 Concrete Lattice Instances

3.1 Flat Lattices

Given any set S, the **flat lattice** $S_\bot^\top$ adds a bottom $\bot$ and top $\top$ with all elements of S incomparable to each other:

```
   top
 / | | \
a  b  c  d  ...
```

```
 \ | | /
   bot
```

Height 2 (three levels including bot/top). Every distinct merge yields $\top$ (contradiction).

Instance	Elements	Application in Prologos
`FlatVal`	bot, val(x), top	General-purpose type inference cells
Constants	bot, {c}, top	Constant propagation
Symbols	bot, sym(s), top	Name resolution, identifier tracking

3.2 Two-Point Lattice (Bool)

The simplest non-trivial lattice: $\{false \sqsubseteq true\}$.

- Join = disjunction (OR)
- Meet = conjunction (AND)
- Used in: reachability analysis, boolean constraint propagation, trait resolution flags.

Implemented as `bool-lattice` in `bilattice.rkt`.

3.3 Three-Point Lattices

3.3.1 Definedness Lattice

$\bot \sqsubseteq \mathit{defined} \sqsubseteq \top$.

Used for tracking whether a value has been computed at all, without caring about its identity.

3.3.2 Belnap's FOUR (Three-Point on Each Axis)

On the knowledge axis: $\bot_k \sqsubseteq t, \bot_k \sqsubseteq f, t \sqsubseteq \top_k, f \sqsubseteq \top_k$.

Four elements, two incomparable pairs. The bilattice of well-founded semantics.

3.4 Powerset Lattices

$(\mathcal{P}(S), \subseteq, \cup, \cap, \emptyset, S)$. A complete Boolean algebra. Height $= |S|$.

Instance	Base Set S	Application in Prologos
Set of constraints	Constraint IDs	Constraint store merging (union)
Set of assumptions	Assumption labels	ATMS environment management
Set of dependencies	Cell/prop IDs	Dependency tracking, worklist management
Set of types	Type constructors	Union type accumulation
Set of trait impls	Instance IDs	Trait resolution candidate sets
Set of capabilities	Capability tokens	Session type capability tracking

3.5 Interval Lattices

3.5.1 Numeric Intervals

$[\ell, u]$ where $\ell \leq u$. Ordered by reverse inclusion: $[a, b] \sqsubseteq [c, d]$ iff $c \leq a$ and $b \leq d$ (narrower = more information). Join = intersection.

- Height: infinite (for real-valued bounds)
- Requires widening/narrowing for termination
- Implemented in Prologos's `lattice.prologos` as the `Interval` type with `Widenable` instance

3.5.2 Universe Level Intervals

Prologos uses universe polymorphism. Level variables form an interval lattice where the merge narrows the range of possible levels. Zonking defaults unsolved levels to `lzero`.

3.6 Map/Dictionary Lattices

Given a key set K and a value lattice V, the map lattice $K \to V$ is ordered pointwise: $m_1 \sqsubseteq m_2$ iff $\forall k. m_1(k) \sqsubseteq m_2(k)$.

Join and meet are computed key-by-key.

Instance	Keys	Values	Application
Type environment	Variable names	Type lattice	Type inference context
Meta solutions	Meta IDs	Term lattice	Unification state
QTT usage map	Variable names	Quantity lattice	Resource consumption tracking
Constraint store	Cell IDs	Constraint sets	Propagator network state
Module registry	Module names	Module contents	Namespace management

Implemented via `map-merge-with` in `lattice.prologos` and hasheq-based registries in `constraint-cell.rkt`.

3.7 Sign Lattice

```
    top
  / | \
neg zero pos
  \ | /
    bot
```

Five elements. Models the sign of a numeric expression. Implemented in `abstract-domains.prologos` as `Lattice Sign`.

3.8 Parity Lattice

```
   top
  /   \
even   odd
  \   /
   bot
```

Four elements. Models even/odd parity. Implemented as `Lattice Parity` in `abstract-domains.prologos`.

3.9 Type Lattice

The lattice over Prologos types, where:

- $\bot$ = unknown (unsolved meta)
- $\top$ = contradiction (inconsistent constraints)
- Concrete types are partially ordered by the subtyping/unification relation

- Compound types (Pi, Sigma, etc.) decompose structurally into sub-lattices

Implemented in `type-lattice.rkt`. Structural decomposition in `elaborator-network.rkt` handles 11 compound type constructors.

3.10 Session Type Lattice

The lattice over session types, where:

- `sess-bot` ($\bot$) = unknown session
- `sess-top` ($\top$) = contradictory session
- Concrete sessions (send, recv, choice, offer, end) merge structurally
- Continuation components merge recursively

Implemented in `session-lattice.rkt`. Behavioral subtyping (safe substitutability of processes) induces the partial order.

3.11 QTT Quantity Lattice

The semiring of quantities $\{0, 1, \omega\}$ with addition and multiplication. Under the information ordering:

- $\bot$ = unknown usage
- 0 = erased at runtime
- 1 = used exactly once (linear)
- ω = unrestricted use

The partial order: $0 \sqsubseteq \omega$ and $1 \sqsubseteq \omega$ (but 0 and 1 are incomparable). This is a bounded join-semilattice.

- **Addition**: resource accumulation ($0 + 1 = 1$, $1 + 1 = \omega$)
- **Multiplication**: scaling ($0 \cdot x = 0$, $\omega \cdot 1 = \omega$)

3.12 Substitution Lattice

The lattice of substitutions (mappings from variables to terms), ordered by generality. The most general unifier (MGU) is the meet of all unifiers. Robinson's unification algorithm computes this meet.

- $\bot$ = the identity substitution (most general)
- $\top$ = failure (no unifier exists)
- Join = anti-unification (least general generalization)
- Meet = unification (most general unifier)

3.13 Herbrand Interpretation Lattice

For logic programming: the powerset of ground atoms, ordered by inclusion. Immediate consequence operator T_P is monotone on this lattice, and its least fixed point gives the minimal Herbrand model.

- **Propagator relevance**: Logic programming in Prologos (unification, resolution) can be modeled as propagation over the Herbrand lattice.

3.14 Constraint Store Lattice

The accumulation of constraints during type checking forms a lattice:

- $\bot$ = empty constraint store
- $\top$ = inconsistent constraints
- Join = union of constraints (more constraints = more information)

Prologos tracks these via `constraint-cell.rkt` with hasheq-based registries using append/union merge strategies.

3.15 ATMS Environment Lattice

Environments in an ATMS (sets of assumptions) form a lattice ordered by subset inclusion. An environment $E_1 \subseteq E_2$ means E_1 is a weaker context.

- Moving **up** the lattice: adding assumptions, making worlds more specific

- **Nogoods**: maximal inconsistent environments
- **Labels**: minimal consistent environments supporting a datum

The ATMS worldview lattice enables simultaneous maintenance of multiple hypothetical contexts without backtracking.

4 Lattice Constructions and Compositions

4.1 Product Lattice

Given lattices $L_1, L_2, \ldots, L_n$, their **Cartesian product** $L_1 \times L_2 \times \cdots \times L_n$ is a lattice with componentwise operations:

$(a_1, \ldots, a_n) \sqcup (b_1, \ldots, b_n) = (a_1 \sqcup_1 b_1, \ldots, a_n \sqcup_n b_n)$

Product lattices are the standard way to track multiple independent aspects of program state simultaneously.

Product	Components	Application
Type × Multiplicity	Type lattice, QTT quantities	Dependent type checking with QTT
Value × Provenance	Value lattice, assumption set	Supported values / ATMS
Lower × Upper	Ascending cell, descending	Bilattice variables
Sign × Parity	Sign lattice, parity lattice	Combined numeric abstract domain
Type × Session	Type lattice, session lattice	Typed communication channels

4.2 Reduced Product

The **reduced product** of lattices L_1 and L_2 is a sublattice of $L_1 \times L_2$ that identifies redundant pairs via a reduction operator ρ. The reduction enforces logical relationships between the components: if knowing $a_1 \in L_1$ constrains what $a_2 \in L_2$ can be, the reduced product captures this.

$\rho(a_1, a_2) = (\alpha_1(\gamma_1(a_1) \cap \gamma_2(a_2)), \alpha_2(\gamma_1(a_1) \cap \gamma_2(a_2)))$

- **Propagator relevance**: Cross-domain propagators (`net-add-cross-domain-propagator`) implement exactly this: when information in one domain constrains another, propagators shuttle the refinement between cells. The reduced product is the implicit lattice of the combined network.

4.3 Sum (Coproduct) Lattices

4.3.1 Disjoint Sum

$L_1 + L_2$: tag elements with their origin, add a shared $\perp$ and $\top$. Elements from different components are incomparable.

4.3.2 Coalesced Sum

$L_1 \oplus L_2$: identify the $\perp$ elements of both lattices. Standard construction in domain theory.

- **Propagator relevance**: Union types in Prologos involve a form of coalesced sum, where different type alternatives share a common bottom (unknown).

4.4 Lifting

$L_\perp$: add a fresh bottom element below all elements of L. Dually, $L^\top$ adds a fresh top.

- $L_\perp$: turns a lattice into one with an explicit "no information" state
- $L^\top$: turns a lattice into one with an explicit "contradiction" state
- $L_\perp^\top$: the standard "flat lattice" construction when L has no pre-existing order

Every propagator cell domain is effectively a lifted-and-topped structure: the raw domain D becomes $D_\perp^\top$ where $\perp$ = nothing and $\top$ = contradiction.

4.5 Smash Product

$L_1 \otimes L_2$: the product with bottoms identified. In domain theory, this corresponds to strict function spaces where $\perp \otimes x = x \otimes \perp = \perp$.

- **Propagator relevance**: When two cells must both have information before any deduction proceeds (strict propagators), the effective domain is a smash product.

4.6 Function Space Lattice

$[L_1 \to L_2]$: the set of monotone functions from L_1 to L_2, ordered pointwise. If L_2 is a complete lattice, so is $[L_1 \to L_2]$.

- **Propagator relevance**: Propagators themselves live in a function space lattice. A propagator is a monotone function $f : L_1 \to L_2$. The space of all such functions forms a lattice, and composition of propagators corresponds to function composition in this lattice.

4.7 Powerdomain Constructions

Powerdomains generalize powersets to domains (lattices with appropriate continuity). Three classical variants:

Powerdomain	Operation	Models	Propagator Application
Hoare (lower)	Union	May-analysis (angelic nondeterminism)	Possible types, reachabl
Smyth (upper)	Intersection	Must-analysis (demonic nondeterminism)	Required properties, gua
Plotkin (convex)	Both	Combined may+must	Full abstract interpretat

- **Propagator relevance**: The Hoare powerdomain models angelic nondeterminism (explored branches in logic programming); the Smyth powerdomain models demonic nondeterminism (all branches must succeed, as in universal quantification over session type choices).

4.8 Galois Connections

A **Galois connection** between posets $(C, \sqsubseteq_C)$ and $(A, \sqsubseteq_A)$ is a pair of monotone functions:

- $\alpha : C \to A$ (abstraction)
- $\gamma : A \to C$ (concretization)

satisfying: $\alpha(c) \sqsubseteq_A a \iff c \sqsubseteq_C \gamma(a)$.

Equivalently, $\alpha \circ \gamma \sqsupseteq id_A$ and $\gamma \circ \alpha \sqsupseteq id_C$ (overconcretizing then abstracting is sound).

4.8.1 Properties

- **Soundness**: abstraction never loses critical information
- **Composability**: $(C \xrightarrow{\alpha_1,\gamma_1} A_1 \xrightarrow{\alpha_2,\gamma_2} A_2)$ gives $(\alpha_2 \circ \alpha_1, \gamma_1 \circ \gamma_2)$
- **Best abstraction**: α gives the most precise element in A that over-approximates c

4.8.2 Galois Connection Catalog for Prologos

Concrete Domain	Abstract Domain	α	γ
Integers $\mathbb{Z}$	Sign lattice	sign extraction	all ints with that sign
Integers $\mathbb{Z}$	Parity lattice	parity extraction	all ints with that parity
Integers $\mathbb{Z}$	Interval $[\ell, u]$	singleton interval	integer range
Concrete types	Type shapes	erase details	all types matching shape
Session protocols	Session skeletons	erase payload types	all sessions matching
Full proof terms	Proof-irrelevant tags	erase proof content	all proofs of proposition
Ground terms	Herbrand abstractions	generalize	instantiate
Concrete states	Abstract states	widened collecting semantics	concrete state sets

Implemented in Prologos as the `GaloisConnection` trait in `lattice.prologos` and cross-domain propagators in `propagator.rkt`.

4.8.3 Composition of Galois Connections

Serial composition: successive layers of abstraction. $C \xrightarrow{(\alpha_1,\gamma_1)} A_1 \xrightarrow{(\alpha_2,\gamma_2)} A_2$ yields $(C, A_2, \alpha_2 \circ \alpha_1, \gamma_1 \circ \gamma_2)$.

Parallel composition: independent aspects of the same concrete domain. $(C, A_1 \times A_2, \langle \alpha_1, \alpha_2 \rangle, \gamma_1 \sqcap \gamma_2)$.

Propagator implementation: serial composition = chain of cross-domain propagators; parallel composition = reduced product with cross-domain propagators maintaining consistency.

4.9 Lattice Embeddings

An **embedding** $e : L_1 \hookrightarrow L_2$ is an injective lattice homomorphism (preserves $\sqcup$ and $\sqcap$). It places L_1 as a sublattice of L_2.

Embedding	From	Into	Application
Bool $\hookrightarrow$ FOUR	2-valued logic	4-valued bilattice	Classical reasoning in WFLE conte
Flat $\hookrightarrow$ Interval	Constants	Numeric intervals	Promoting constant to range
Sign $\hookrightarrow$ Interval	Sign domain	Interval domain	Refining sign with range bounds
Type $\hookrightarrow$ Type $\times$ Q	Types	Types + quantities	Adding QTT annotations
Local session $\hookrightarrow$ Global	Endpoint types	Multiparty protocol	Session type projection

Embeddings enable incremental enrichment of cell domains: start with a simple lattice and promote to a richer one as more information arrives.

4.10 Lattice Quotients

Given a lattice L and a congruence relation $\equiv$ (an equivalence compatible with $\sqcup$ and $\sqcap$), the quotient $L/\equiv$ is a lattice.

- **Propagator relevance**: Alpha-equivalence on terms, eta-equivalence, and definitional equality all define congruences on the type lattice. Zonking (`zonk.rkt`) can be understood as computing in a quotient lattice where solved metas are identified with their solutions.

4.11 Lattice Fixed-Point Operators

4.11.1 Knaster-Tarski Theorem

Every monotone function $f : L \to L$ on a complete lattice L has:

- A **least fixed point** $\mu f = \{x \mid f(x) \sqsubseteq x\}$
- A **greatest fixed point** $\nu f = \bigsqcup\{x \mid x \sqsubseteq f(x)\}$

4.11.2 Kleene Iteration

$\bot, f(\bot), f^2(\bot), \ldots$ converges to μf when L has finite height or f is \$$\omega$\$-continuous.

4.11.3 Widening and Narrowing

For infinite-height lattices, the Kleene chain may not converge. The Cousot solution:

- **Widening** ∇: accelerates convergence by jumping past intermediate values. $a \nabla b \sqsupseteq a \sqcup b$. Guarantees termination but overshoots.

- **Narrowing** Δ: recovers precision. $a \sqcap b \sqsubseteq a \Delta b \sqsubseteq a$. Iteratively tightens the overapproximation.

Implemented via the `Widenable` trait in `lattice.prologos` and `widen-fns` in `propagator.rkt`.

4.11.4 Stratified Fixed Points

For logic programs with negation, the standard model is computed via **stratification**: partition predicates into strata where negation only refers to lower strata, compute fixed points bottom-up stratum by stratum.

Prologos implements this via the stratified quiescence protocol in the propagator migration, where constraint resolution proceeds in ordered phases.

4.12 Lattice-Indexed Families

A family of lattices $\{L_i\}_{i \in I}$ indexed by some set I. The **dependent product** $\prod_{i \in I} L_i$ (pointwise operations) generalizes the simple product. The **dependent sum** $\sum_{i \in I} L_i$ (tagged union) generalizes the coproduct.

- **Propagator relevance**: In dependent type theory, the type of a later argument may depend on the value of an earlier one. This creates dependent lattice structures: the lattice used for a cell may depend on the resolved value of another cell. Prologos handles this via structural decomposition, which lazily creates sub-cells whose domains depend on the compound type being decomposed.

5 Combinatorial Lattice Application Map

This section maps how lattice structures combine to serve each major Prologos subsystem.

5.1 Type Inference Engine

Component	Lattice Structure	Composition
Meta-variables	Flat lattice (unresolved/solved/conflict)	Map lattice over meta IDs
Type terms	Structural type lattice with decomposition	Dependent product of sub-ce
Constraint accumulation	Powerset lattice of constraints	Union merge
Trait resolution	Powerset lattice of candidate instances	Intersection for disambiguati
Universe levels	Interval lattice $[\ell, u]$	Product with type lattice

Cross-cutting: Galois connection from concrete types to type shapes for speculative checking (`save-meta-state` / `restore-meta-state!`).

5.2 QTT Resource Tracking

Component	Lattice Structure	Composition
Usage quantities	$\{0, 1, \omega\}$ semiring-lattice	Map lattice over variable names
Multiplicity checking	Two-point (pass/fail) lattice	Product with type environment
Dict param handling	Quantity lattice (mw not m0)	Embedded in type context

The semiring structure (addition + multiplication) sits atop the lattice ordering, forming a structure akin to a quantale.

5.3 Session Types

Component	Lattice Structure	Composition
Session skeletons	Session type lattice (send/recv/choice/end)	Structural decomposition
Channel capabilities	Powerset lattice of capabilities	Product with session lattice
Protocol conformance	Two-point (conformant/non-conformant)	Galois connection from sessions
Multiparty projection	Product of endpoint session lattices	Embedding from local to globa
Duality checking	Complemented lattice on session pairs	Product with session lattice

5.4 Well-Founded Logic Engine (WFLE)

Component	Lattice Structure	Composition
Truth values	Belnap FOUR bilattice	Product (truth $\times$ knowledge)
Bilattice variables	Lower $\times$ Upper (ascending $\times$ descending)	Reduced product with consistency
Predicate extensions	Powerset of ground atoms	Map lattice over predicate names
Negation	Antitone Galois connection on bilattice	Composes with positive fixpoint
Stratification	Totally ordered lattice of strata	Index for lattice-indexed family

5.5 Abstract Interpretation Framework

Component	Lattice Structure	Composition
Sign analysis	5-element sign lattice	Galois connection from integers
Parity analysis	4-element parity lattice	Galois connection from integers
Interval analysis	Interval lattice with widening	Galois connection from integers
Combined numeric	Reduced product (sign × interval)	Cross-domain propagators
Pointer analysis	Powerset lattice of abstract locations	Map lattice over variables
Constant propagation	Flat lattice over constant values	Map lattice over variables
Reachability	Two-point lattice	Powerset over program points

5.6 ATMS and Search

Component	Lattice Structure	Composition
Environments	Powerset lattice of assumptions	Ordered by subset inclusion
Supported values	Product (value × environment set)	Value lattice × ATMS lattice
Nogood detection	Boolean lattice (consistent/inconsistent)	Map over environment lattice
Worldview management	Lattice of environments modulo nogoods	Quotient lattice
Dependency tracking	Powerset lattice of justification IDs	Map over datum lattice

5.7 Propagator Network Infrastructure

Component	Lattice Structure	Composition
Cell values	Domain-specific (parameterized)	Indexed family over cell IDs
Worklist	Powerset of propagator IDs	Priority-ordered
Network state	Product of all cell lattices	Massive product lattice
Fuel counter	Descending natural numbers	Separate from value lattice
Contradiction flag	Two-point lattice	Join across all cells
Trace/observability	Sequence lattice (append-only log)	Product with network state

6 Advanced Composition Patterns

6.1 Lattice Transformers

By analogy with monad transformers, we can define **lattice transformers** that systematically enrich a base lattice:

Transformer	Type	Effect
`Lift_\bot`	$L \mapsto L_\bot$	Adds explicit "no information" bottom
`Lift^\top`	$L \mapsto L^\top$	Adds explicit "contradiction" top
`Flat`	$S \mapsto S_\bot^\top$	Flattens a set into a lattice
`Pow`	$L \mapsto \mathcal{P}(L)$	Powerset over lattice elements
`Map_K`	$L \mapsto (K \to L)$	Pointwise extension over key set
`Interval`	$L \mapsto [L, L]$	Interval abstraction (requires total order)
`Support`	$L \mapsto L \times \mathcal{P}(A)$	Adds provenance/assumption tracking
`Bilat`	$L \mapsto L^{asc} \times L^{desc}$	Bilattice pairing
`Widen`	$L \mapsto L + \nabla$	Adds widening operator for convergence

These compose: `Widen(Map_K(Interval(\mathbb{Z})))` gives a widened map of integer intervals keyed by variable name—an abstract domain for interval analysis.

6.2 Fibered Lattices

When the lattice structure at each point depends on the value at another point, we have a **fibered** or **dependent** lattice. In Prologos:

- The type of a function body depends on the type of its argument (dependent function types / Pi types)
- The session type of a continuation depends on the choice made at a branch (session type choice)
- The decomposition sub-cells created depend on the head constructor of the compound type (structural decomposition)

These are modeled by **fibrations** in category theory: a functor $p : \mathbf{E} \to \mathbf{B}$ where the fiber $p^{-1}(b)$ over each object b in the base category is a lattice.

6.3 Lattice Comonads and Demand

The dual of lifting (adding bottom) is **colift** or **demand**: annotating each lattice element with how urgently it is needed. In functional-logic programming (narrowing), demand drives which positions to narrow first.

- **Needed narrowing** (Antoy/Hanus) uses definitional trees to identify demanded positions

- This corresponds to a comonadic structure on the lattice: `extract` gives the current value, `extend` propagates demand downward through term structure

Prologos's FL narrowing design (`FL_NARROWING_DESIGN.org`) connects this to the propagator scheduler's worklist priorities.

6.4 Lattice Adjunctions Beyond Galois Connections

Galois connections are adjunctions between poset categories. More general adjunctions (between lattice-valued functors) enable:

- **Free/forgetful adjunctions**: the free lattice over a poset, the forgetful functor from lattices to posets. Relevant to generating lattice structure from user-defined types.
- **Kan extensions**: generalizing Galois connections to situations where the domains are not directly comparable but related through a third structure.
- **Profunctor optics**: lattice-valued lenses for focusing on sub-components of compound lattice elements (connects to structural decomposition).

7 Speculative and Creative Applications

7.1 Lattice of Lattices

The class of all finite lattices, ordered by embeddability, itself has lattice-like structure. This meta-lattice could inform:

- **Domain inference**: automatically selecting the right lattice for a cell based on the constraints feeding into it
- **Lattice migration**: promoting a cell from a simpler lattice to a richer one as analysis demands grow (e.g., flat $\rightarrow$ interval $\rightarrow$ polyhedra)

7.2 Tropical Semiring Lattice

The **tropical semiring** $(\mathbb{R} \cup \{+\infty\}, \min, +)$ is a semiring where "addition" is min and "multiplication" is $+$. This forms a lattice under the min ordering.

- Application: shortest-path analysis, optimality propagation, cost-aware type checking (e.g., choosing the cheapest trait instance).

7.3 Partition Lattice

The lattice of all partitions of a set, ordered by refinement. Meet = finest common coarsening, join = coarsest common refinement.

- Application: equivalence class management in unification, union-find viewed as traversal of the partition lattice.

7.4 Concept Lattice (Formal Concept Analysis)

Given a set of objects G, attributes M, and incidence relation $I \subseteq G \times M$, the **concept lattice** $\mathfrak{B}(G, M, I)$ consists of formal concepts (maximal rectangles in the incidence matrix).

- Application: discovering implicit structure in trait/type relationships, automatic classification of types by their supported operations.

7.5 Information Flow Lattice

A lattice of security levels (e.g., unclassified $\sqsubseteq$ confidential $\sqsubseteq$ secret $\sqsubseteq$ top-secret) used in information flow analysis.

- Application: could model capability levels in Prologos's session types, ensuring that session interactions respect capability hierarchies. Also relevant to any future effect-system or region-based resource tracking.

7.6 Lattice of Regular Languages

Regular languages over an alphabet Σ, ordered by inclusion, form a lattice (closed under union and intersection).

- Application: session type protocols as regular languages, where protocol conformance = language inclusion. Merge of session constraints = intersection of accepted protocol languages.

7.7 Scott Information Systems

An information system $(A, Con, \vdash)$ where A is a set of tokens, Con is the set of consistent subsets, and $\vdash$ is the entailment relation. The elements of the associated Scott domain are the ideals (downward-closed, directed, consistent subsets).

- Application: a general framework for defining the information content of cells. Each cell's domain could be specified as an information system, with the propagator network implementing the entailment relation.

7.8 Residuated Lattices

A lattice with an additional monoid operation and left/right residuals (generalizing division). Residuated lattices are the algebraic semantics of substructural logics (relevant, linear, etc.).

- Application: direct algebraic model of Prologos's linear type system. The residuation $a \backslash b$ (the largest x such that $a \otimes x \sqsubseteq b$) models linear implication.

8 Summary Table: Lattice Applicability to Prologos Subsystems

Lattice	Type Inf	QTT	Sessions	WFLE	AbsInt	ATMS	Logic	Narrowin
Flat	X				X			
Bool (2-point)	X			X		X	X	
Powerset	X		X	X	X	X	X	
Interval					X			
Map/Dict	X	X		X	X	X		
Sign					X			
Parity					X			
Bilattice (FOUR)				X			X	
Type lattice	X							
Session lattice			X					
QTT semiring		X						
Substitution	X						X	X
Herbrand							X	X
Constraint store	X		X	X				
ATMS environment						X		
Product (general)	X	X	X	X	X	X		
Reduced product					X			
Galois connection			X	X	X		X	
Function space	X				X			
Heyting algebra	X						X	
Residuated lattice		X						
Partition lattice	X							
Tropical semiring								
Concept lattice	X							
Regular language lattice			X					

9 References

- Radul, A. (2009). *Propagation Networks: A Flexible and Expressive Substrate for Computation.* PhD thesis, MIT.
- Sussman, G. J. & Radul, A. (2009). "The Art of the Propagator." MIT AI Memo.
- Cousot, P. & Cousot, R. (1977). "Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints." *POPL.*
- Cousot, P. & Cousot, R. (2014). "A Galois Connection Calculus for Abstract Interpretation." *POPL.*

- Kuper, L. (2014). *Lattice-based Data Structures for Deterministic Parallel and Distributed Programming*. PhD thesis, Indiana University.
- Belnap, N. (1977). "A Useful Four-Valued Logic." In *Modern Uses of Multiple-Valued Logic*, Reidel.
- Fitting, M. (1991). "Bilattices and the Semantics of Logic Programming." *Journal of Logic Programming*.
- Davey, B. A. & Priestley, H. A. (2002). *Introduction to Lattices and Order*. Cambridge University Press.
- Garg, V. K. (2015). *Introduction to Lattice Theory with Computer Science Applications*. Wiley.
- Atkey, R. (2018). "Syntax and Semantics of Quantitative Type Theory." *LICS*.
- de Kleer, J. (1986). "An Assumption-based TMS." *Artificial Intelligence*.
- Abramsky, S. & Jung, A. (1994). "Domain Theory." In *Handbook of Logic in Computer Science*, Oxford University Press.
- Knaster, B. (1928) and Tarski, A. (1955). "A Lattice-Theoretical Fixpoint Theorem and its Applications." *Pacific Journal of Mathematics*.